



SKILL WORKBOOK

**24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L FULL STACK APPLICATION
DEVELOPMENT**

TEAM FSAD
K L UNIVERSITY | VADDESWARAM

SKILL WORKBOOK

24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L - FSAD

STUDENT NAME	Manan Shah
STUDENT ID	2400090137
YEAR	2nd
SEMESTER	EVEN
FACULTY NAME	Sridevi Sakhamuri

DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L FULL STACK
APPLICATION DEVELOPMENT

Date of the Session: ____/____/____

Time of The Session: ____to____

#SKILL - 3 → Working with HQL - Sorting, Pagination & Aggregates

Prerequisites:

- **Basic knowledge of Hibernate**
- **Basic idea of HQL syntax**

In this lab, you will extend the Hibernate module created in **Skill 2** and perform advanced data retrieval using **HQL**, including **sorting**, **pagination**, **filtering**, and **aggregate functions**. These operations help in retrieving structured and meaningful data from the database using Hibernate.

Tasks:

1. Use the **Product entity** created in Skill 2.
2. Add **5–8 additional Product records** into the database.
3. Write HQL queries to retrieve all products sorted by price:
 - a. Ascending order
 - b. Descending order
4. Write an HQL query to sort products by quantity (highest first).
5. Implement pagination using HQL to display:
 - a. First 3 products
 - b. Next 3 products
6. Write HQL queries for aggregate operations:
 - a. Count total number of products
 - b. Count products where quantity > 0
 - c. Count products grouped by description
 - d. Find minimum and maximum price
7. Write an HQL query using **GROUP BY** to group products by description.
8. Write an HQL query using **WHERE** to filter products within a price range.
9. Write HQL queries using **LIKE** such as:
 - a. Names **starting with** certain letters
 - b. Names **ending with** certain letters
 - c. Names containing a pattern **anywhere** (substring)
 - d. Names with an **exact character length**
10. Push the Hibernate project to GitHub.

Com.klu.app

HQLDemoApp.java

```
package com.klu.app;
```

```
import java.util.List;
```

```
import org.hibernate.Query;
```

```
import org.hibernate.Session;
```

```
import com.klu.entity.Product;
```

```
import com.klu.util.HibernateUtil;
```

```
public class HQLDemoApp {
```

```

public static void main(String[] args) {

    Session session = HibernateUtil.getSessionFactory().openSession();

    // Sort by price ASC
    List<Product> list1 =
        session.createQuery("FROM Product p ORDER BY p.price ASC").list();
    list1.forEach(System.out::println);

    // Sort by quantity DESC
    session.createQuery("FROM Product p ORDER BY p.quantity DESC").list();

    // Pagination
    Query q = session.createQuery("FROM Product");
    q.setFirstResult(0);
    q.setMaxResults(3);
    q.list();

    session.close();
    System.out.println("HQL executed successfully");
}
}

```

InsertProducts.java

```

package com.klu.app;

import org.hibernate.Session;
import org.hibernate.Transaction;

import com.klu.entity.Product;
import com.klu.util.HibernateUtil;

public class InsertProducts {

    public static void main(String[] args) {

        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();

```

```

        session.save(new Product("Laptop", "Electronics", 55000, 5));
        session.save(new Product("Mouse", "Electronics", 500, 20));
        session.save(new Product("Keyboard", "Electronics", 1500, 15));
        session.save(new Product("Chair", "Furniture", 3000, 10));
        session.save(new Product("Table", "Furniture", 7000, 3));
        session.save(new Product("Bottle", "Kitchen", 800, 25));

        tx.commit();
        session.close();

        System.out.println("Products inserted successfully");
    }
}

```

Com.klu.entity

Product.java

```

package com.klu.entity;

import javax.persistence.*;

@Entity
@Table(name = "product")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;
    private String description;
    private double price;
    private int quantity;

    public Product() {}

    public Product(String name, String description, double price, int quantity) {
        this.name = name;

```

```

        this.description = description;
        this.price = price;
        this.quantity = quantity;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getDescription() { return description; }
    public double getPrice() { return price; }
    public int getQuantity() { return quantity; }
}

```

Com.klu.util

HibernateUtil.java

```

package com.klu.util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static SessionFactory factory;

    static {
        try {
            factory = new Configuration()
                .configure("hibernate.cfg.xml")
                .buildSessionFactory();
            System.out.println("SessionFactory created successfully");
        } catch (Exception e) {
            System.err.println("SessionFactory creation failed");
            e.printStackTrace();
        }
    }

    public static SessionFactory getSessionFactory() {
        return factory;
    }
}

```

Src/main/java

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<hibernate-configuration>
  <session-factory>

    <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver
    </property>

    <property name="hibernate.connection.url">
      jdbc:mysql://localhost:3306/inventorydb
    </property>

    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">Ashu@2005</property>

    <property name="hibernate.dialect">
      org.hibernate.dialect.MySQL8Dialect
    </property>

    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>

    <mapping class="com.klu.entity.Product"/>

  </session-factory>
</hibernate-configuration>
```

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
  4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.klu</groupId>
  <artifactId>Experiment-3</artifactId>
  TEAMFSAD
```



```

    <version>0.0.1-SNAPSHOT</version>
    <dependencies>
<!-- Hibernate Core -->
    <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>5.6.15.Final</version>
    </dependency>
<!-- MySQL Driver -->
    <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
    </dependency>
<!-- JPA -->
    <dependency>
    <groupId>javax.persistence</groupId>
    <artifactId>javax.persistence-api</artifactId>
    <version>2.2</version>
    </dependency>
    </dependencies>
</project>

```

```

mysql> use inventorydb;
Database changed
mysql> show tables;
+-----+
| Tables_in_inventorydb |
+-----+
| product                |
+-----+
1 row in set (0.05 sec)

mysql> |

```

FSAD WORKBOOK - Experiment-3/src/main/java/com/klu/app/HQLDemoApp.java - Spring Tools for Eclipse

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- Maven Dependencies
- src
- target
- pom.xml
- Experiment-3 [FSAD WORKBOOK main]
- src/main/java
 - com.klu.app
 - HQLDemoApp.java
 - InsertProducts.java
 - com.klu.entity
 - Product.java
 - com.klu.util
 - HibernateUtil.java
- src/main/resources
 - hibernate.cfg.xml
- src/test/java
- src/test/resources
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target
- pom.xml

hibernate.cfg.xml

```
1 package com.klu.app;
```

HQLDemoApp.java

```
1 package com.klu.app;
```

Console

```
terminated: HQLDemoApp [Java Application] C:\Program Files (x86)\Java\jre1.8.0_471\bin\javaw.exe (Feb 6, 2026, 10:20:29 PM - 10:20:34 PM elapsed 0.0005) [pid: 15656]  
INFO: HH10001005: using driver [com.mysql.cj.jdbc.Driver] at URL [jdbc:mysql://localhost:3306/inventorydb]  
Feb 06, 2026 10:20:33 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator  
INFO: HH10001001: Connection properties: {user=root, password=****}  
Feb 06, 2026 10:20:33 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator  
INFO: HH10001001: Connection properties: {user=root, password=****}  
Feb 06, 2026 10:20:33 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator  
INFO: HH10001003: Autocommit mode: false  
Feb 06, 2026 10:20:33 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator  
INFO: HH10001003: Autocommit mode: false  
Feb 06, 2026 10:20:33 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator  
INFO: HH1000115: Hibernate connection pool size: 20 (min=1)  
Feb 06, 2026 10:20:33 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator  
INFO: HH1000115: Hibernate connection pool size: 20 (min=1)  
Feb 06, 2026 10:20:34 PM org.hibernate.dialect.Dialect <init>  
INFO: HH1000400: Using dialect: org.hibernate.dialect.MySQL8Dialect  
Feb 06, 2026 10:20:34 PM org.hibernate.dialect.Dialect <init>  
INFO: HH1000400: Using dialect: org.hibernate.dialect.MySQL8Dialect  
Feb 06, 2026 10:20:34 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection  
INFO: HH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJdbcConn  
Feb 06, 2026 10:20:34 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection  
INFO: HH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJdbcConn  
Feb 06, 2026 10:20:34 PM org.hibernate.engine.transaction.jta.platform.internal.JtaPlatformInitiator initiateService  
INFO: HH1000490: Using JtaPlatform implementation: [org.hibernate.engine.transaction.jta.platform.internal.NoJtaPlatform]  
Feb 06, 2026 10:20:34 PM org.hibernate.engine.transaction.jta.platform.internal.JtaPlatformInitiator initiateService  
INFO: HH1000490: Using JtaPlatform implementation: [org.hibernate.engine.transaction.jta.platform.internal.NoJtaPlatform]  
SessionFactory created successfully  
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_, product0_.quantity  
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_, product0_.quantity  
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_, product0_.quantity  
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_, product0_.quantity  
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_, product0_.quantity  
HQL executed successfully  
HQL executed successfully
```

Boot Dashboard

Type tags, projects, or working set names to match (incl. * and ? wildc

local

Writeable Smart Insert 16:1:335

VIVA QUESTIONS:

1. What is HQL and how does it differ from SQL?

HQL (Hibernate Query Language) is a query language used in Hibernate to retrieve data from the database using Java class names and properties instead of table names and column names. The main difference is that SQL works directly with database tables, while HQL works with objects and is database-independent, making it more suitable for Java applications using Hibernate.

2. Why do we use sorting in HQL?

Sorting in HQL is used to arrange the retrieved data in a specific order, such as ascending or descending. It helps in organizing results based on a particular field like name, date, or salary, making the output easier to read and analyze.

3. What is pagination and where is it used?

Pagination is the process of dividing large sets of data into smaller parts (pages). It is commonly used in web applications, such as search results or product listings, where displaying all records at once would be slow and difficult to manage. Hibernate supports pagination using methods like `setFirstResult()` and `setMaxResults()`.

4. Name any three aggregate functions supported in HQL.

Three aggregate functions supported in HQL are **COUNT()**, **SUM()**, and **AVG()**. These functions are used to perform calculations on groups of data.

5. What is the difference between HQL and HCQL (Hibernate Criteria Query Language)?

HQL is a string-based query language where queries are written manually as text, similar to SQL. HCQL (Criteria Query Language) is a programmatic way of creating queries using Java objects and methods instead of writing query strings. HQL is easier for simple queries, while HCQL is useful for dynamic queries where conditions change at runtime.

(For Evaluator's use only)

<u>Comment of the Evaluator (if Any)</u> 	<u>Evaluator's Observation</u> Marks Secured: _____ out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:
--	---

